

Final Project: gpt-oss

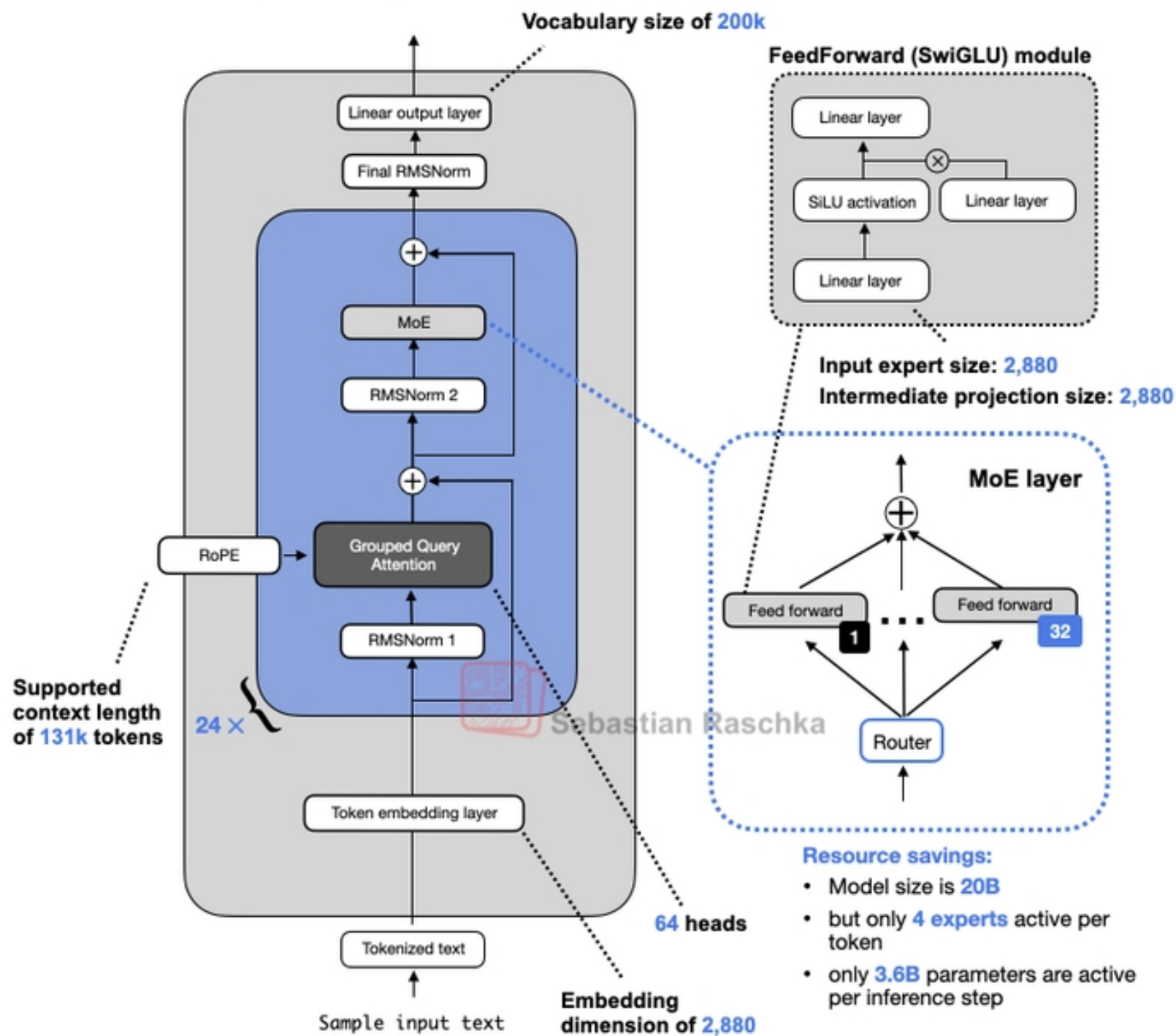
Team 07,

Oct 6, 2025.

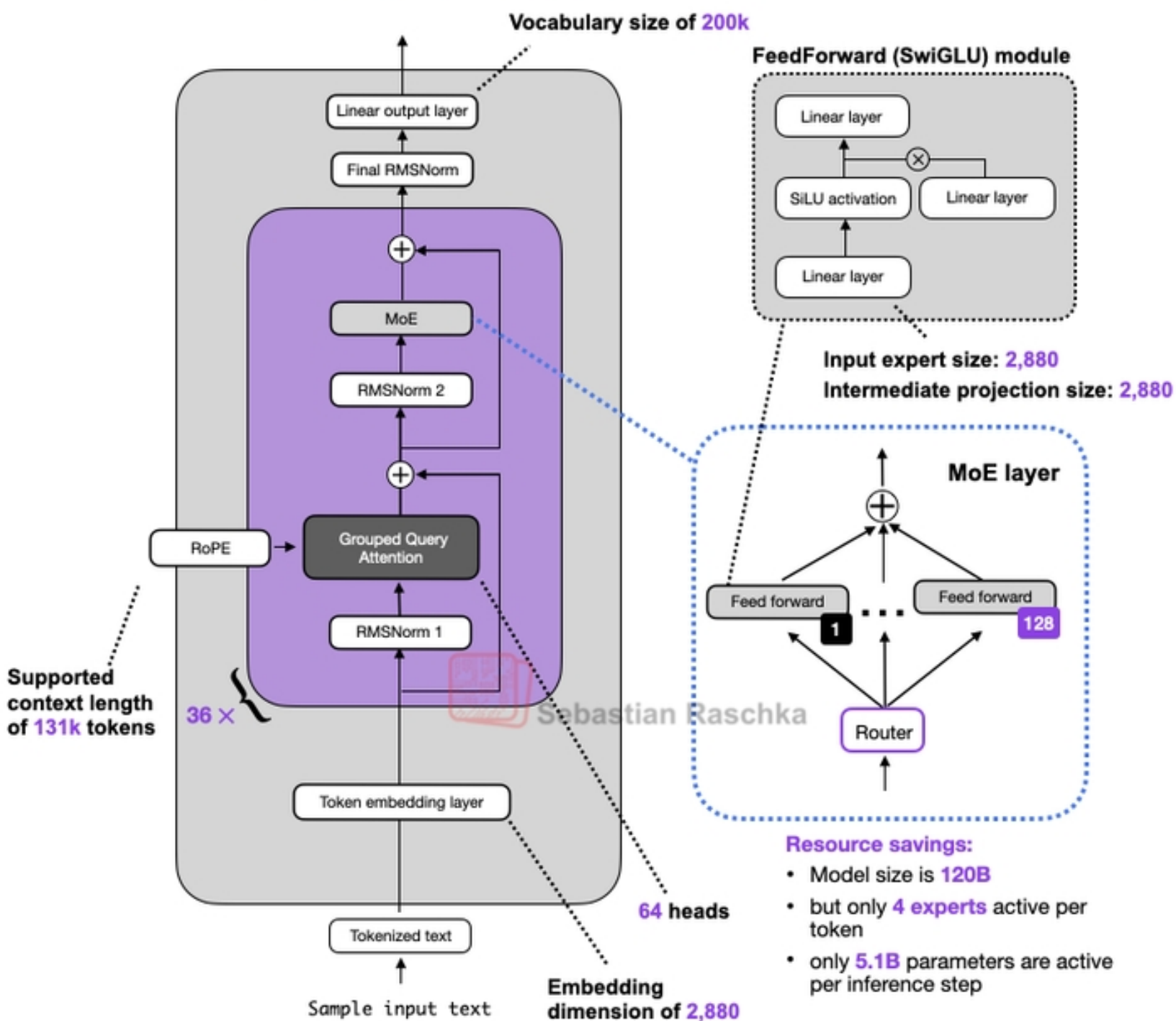


Overall Architecture

GPT-OSS 20B



GPT-OSS 120B



Setup

- Model 20B:
 - #inputs: 12288
 - #GPUs: 8
 - Expected runtime: 5-6 mins
- Model 120B:
 - #inputs: 14848
 - #GPUs: 8
 - Expected runtime: 20 mins

Batching & Scheduler

- Increasing batch size often leads to the rise of throughput:
 - Increase resource utilization
 - Reduce overhead: launch kernel, synchronization, ...
 - Optimize access pattern
 - Eliminate padding for mfma matmul
- Continuous batching: often better when outputs tend to terminate early.

T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1				
S_2	S_2	S_2					
S_3	S_3	S_3	S_3				
S_4	S_4	S_4	S_4	S_4			

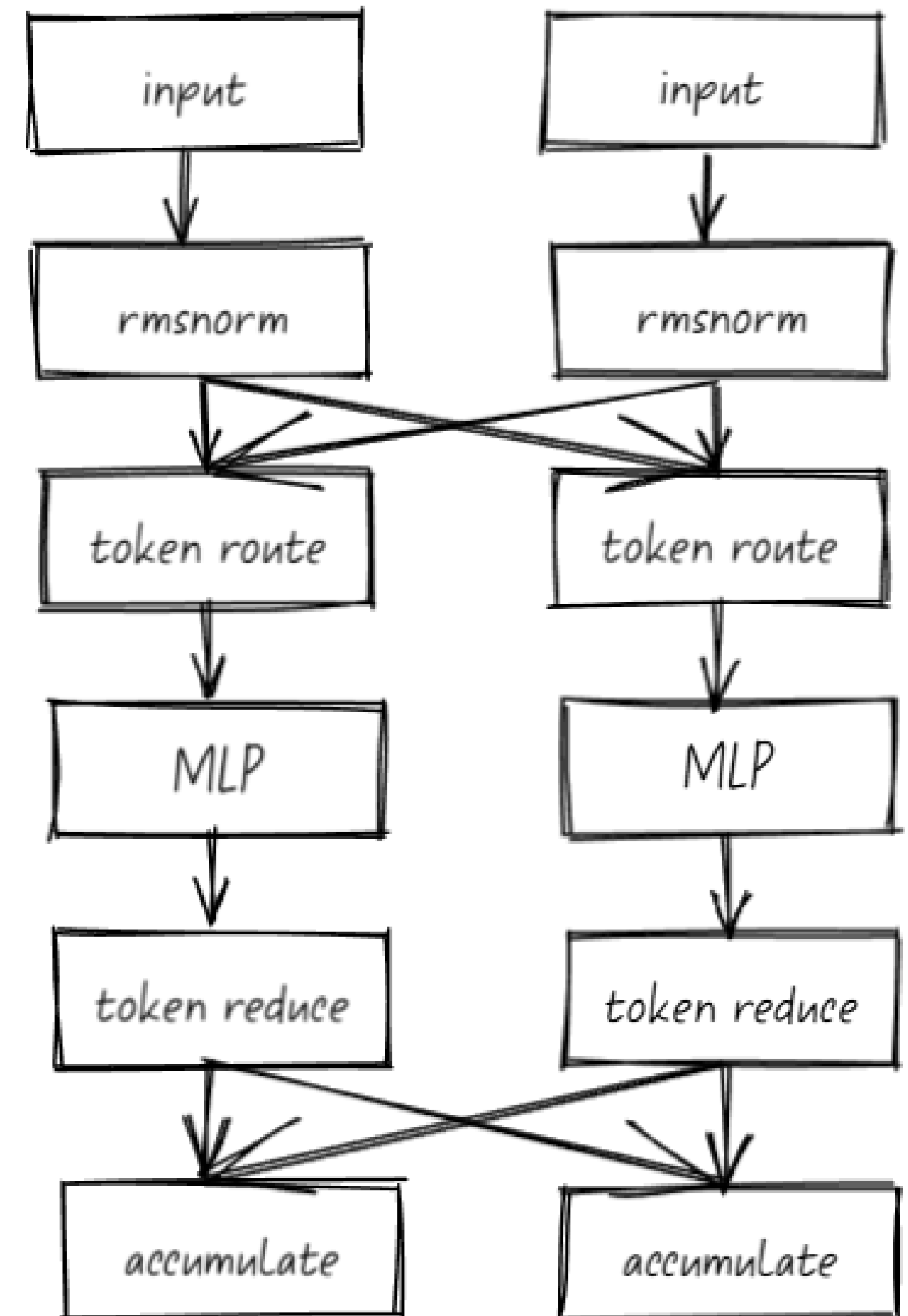
T_1	T_2	T_3	T_4	T_5	T_6	T_7	T_8
S_1	S_1	S_1	S_1	S_1	END	S_6	S_6
S_2	S_2	S_2	S_2	S_2	S_2	S_2	END
S_3	S_3	S_3	S_3	END	S_5	S_5	S_5
S_4	S_4	S_4	S_4	S_4	S_4	END	S_7

Data Parallelism

- Divide by input prompts
- Individual GPU handles independently and gather results at the end
- Pros: less communication overhead, easy to implement
- Cons: cannot reduce memory manage → limits batch size

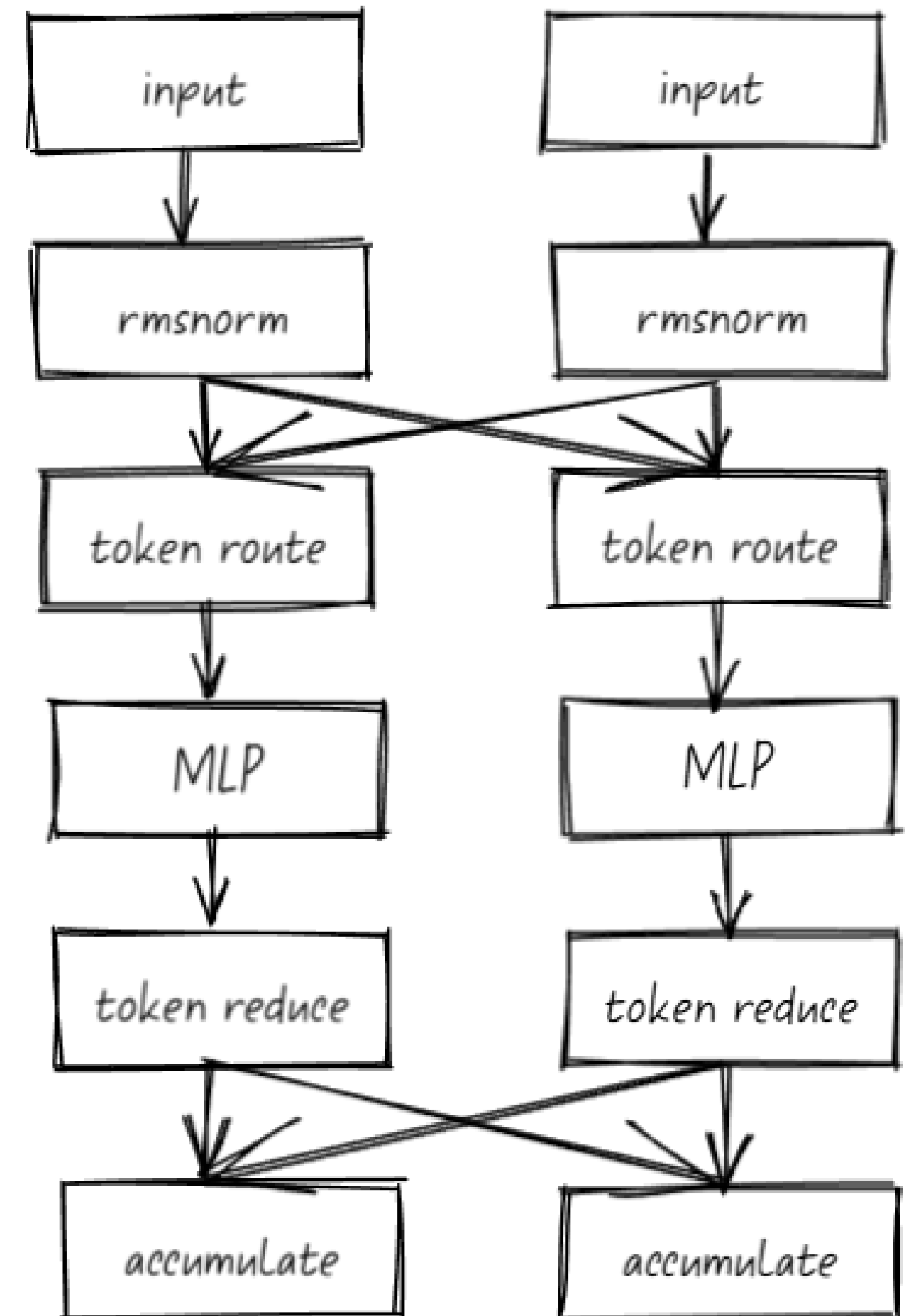
Tensor Parallelism

- Only parallelize for MoE
 - Divide weights of MLP1 và MLP2:
 - w_{mlp1} : [layers, experts, $2 * D$, H] $\rightarrow$ [layers, experts, $2 * D / p$, H]
 - w_{mlp2} : [layers, experts, H , D] $\rightarrow$ [layers, experts, H , D / p]
 - Overall flow:
 - Each gpu processes a set of independent prompts (similar to DP)
 - All-gather after rmsnorm
 - Reduce-scatter after MLPs



Tensor Parallelism

- Trade-off: time vs memory
 - Do not apply TP for attention as memory usage of attention weight is much smaller than MoE → not worthy.
 - On each gpu, M-dimension (batch size) increases by TP_SIZE times while N-dimension and K-dimension decrease by TP_SIZE times in MLP1 and MLP2, respectively.
 - Set TP_SIZE small (2-4) to guarantee the gain by increasing batch size pass overhead.



Tensor Parallelism

- Pros:

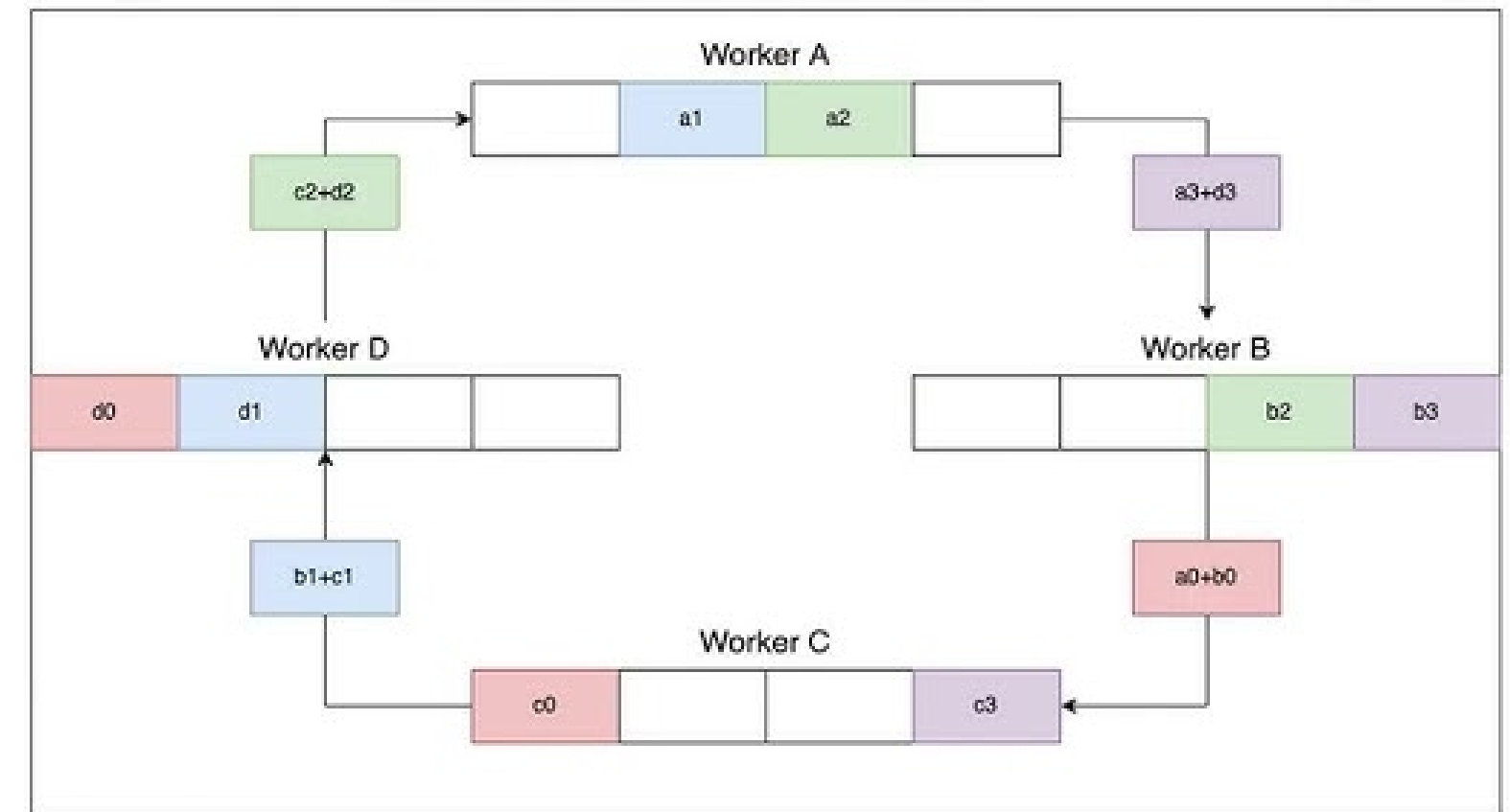
- Saves memory compared to DP
- No imbalanced workload like EP
- No bubble hole like PP
- Increases M-dimension (often small) for matmul MLP in MoE phase

- Cons:

- Communication overhead rises significantly when increasing TP SIZE
- More communication than EP (all-to-all instead of peer-to-peer)
- Decreases N, K-dimension → can reduce throughput in case $M \gg N, K$

Ring Topology

- all-gather:
 - Iterate $p-1$ times, rank r pulls new data from rank $(r-1) \% p$
- reduce-scatter:
 - Iterate $p-1$ times, rank r pulls chunk r from rank $(r+i) \% p$ and performs reduction
- Using bf16 instead of fp32 for communication → reduce packet size → increase throughput



Ring Topology

- Pros:
 - Balances workload compared to Master-Slave, all links are busy, no bottleneck
 - In each iteration, each GPU only sends to and receives from 1 GPU
- Cons
 - Latency increases when using many GPUs
 - Large complexity compared to recursive doubling

Parallel Architecture

- 20B:
 - 2 TP (naive) + 4 DP
 - Batch size: 1536
- 120B:
 - 4 TP (ring) + 2 DP
 - Batch size: 1856
- Balance between throughput gained by increasing batch size and declined by communication overhead

Memory optimization

- MLP weights:
 - Tensor parallelism
 - MXFP4 quantization for MLP weights (uint8 scale)
- Run state:
 - Store all runstate in bf16 format
 - Using KV Cache INT8 for 120B (first 16 elements are kept in BF16 to keep accuracy)

Weights

Thành phần chính	Shape	20B (TP = 2, MoE bf16)	120B (TP = 4, MoE MXFP4)
Embedding (BF16)	[V, H]	1.08	1.08
WQKV (BF16)	[L, h * (n_heads + 2 * n_kv_heads), H]	0.66	0.99
WO (BF16)	[L, H, H]	0.53	0.79
Output head (BF16)	[V, H]	1.08	1.08
Router weights (BF16)	[L,H,n_experts]	0.004	0.025
MoE w_mlp1	[L,n_layers, n_experts, 2*I, H]	11.87 (BF16) / GPU	9.452 (MXFP4)
MoE w_mlp2	[L,n_layers, n_experts, H, I]	5.93 (BF16) / GPU	4.726 (MXFP4)

MXFP4 Quantization

- Model 120B: 36 layers, each layer has 128 experts
- MXFP4 compresses each element:
 - 4-bit data $\rightarrow$ 0.5 B/elem
 - A common scale uint8 for 32 elements $\rightarrow$ 1/32 B/elem
 - $\Rightarrow 0.5 + 1/32 = 0.53125$ B/elem
- MLP weights on 1 GPU:
 - w_mlp1 (Gate+Up) $= L * n_experts * 2I * H * 0.53125 / TP = 9.452$ GiB
 - w_mlp2 (Down): $[H,I] = L * n_experts * I * H * 0.53125 / TP = 4.726$ GiB
- Drawbacks: As hardware does not support direct calculation on MXFP4 $\rightarrow$ need to dequantize back into BF16 or FP32 $\rightarrow$ overhead

RunState (KV Cache 20B)

- Use bf16 format
- Memory usage :
 - Odd layer = $\text{BATCH_SIZE} * L * \text{MAX_SEQ_LEN} * 8 * \text{sizeof}(\text{bf16})$
 - Even layer = $\text{BATCH_SIZE} * L * \text{SW_SIZE} * 8 * \text{sizeof}(\text{bf16})$
 - With $\text{MAX_SEQ_LEN} = 1024$, $\text{SW_SIZE} = 128 \rightarrow 20\text{B} : \text{BATCH_SIZE} = 1536$
- Note: for even layers, only need to allocate K/V buffer for $\text{SW_SIZE} = 128$ tokens instead of full (1024 tokens) and replace on elements outside sliding window

RunState (KV Cache 120B)

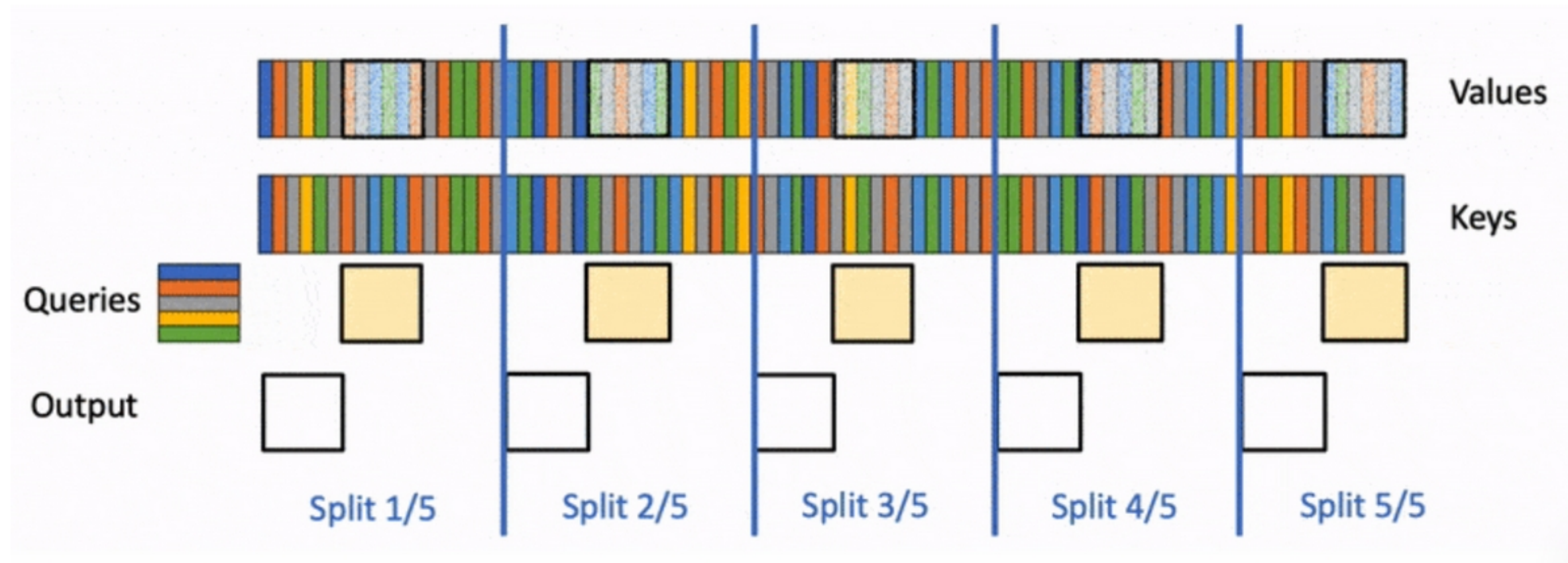
- KV cache int8 with scales for model 120B
 - **Quantize:**
 - k_cache of 1 token has $8 * 64 = 512$ elements
 - Find **max_k** on these 512 elements and quantize each using this formula: **$\text{int8_x} = \text{clamp}(\text{round}(x * 127 / \text{max_k}), -127, 127)$**
 - **Dequantize:**
 - **$x = \text{float}(\text{int8_x}) * \text{max_k} / 127$**
 - **Mix precision:** Keep the first 16 tokens in bf16 to ensure accuracy
 - int8 with scales give acceptable accuracy and lower dequantize cost than **fp8**
 - Batch size 120B model: 1856

Attention

- When calculating attention in a decoding step for BATCH_SIZE samples:
 - gpt-oss uses grouped query attention: 8 queries use a common 1 kv head
 - Even layers use sliding window = 128
 - Odd layers apply attention for full sequence length 1024

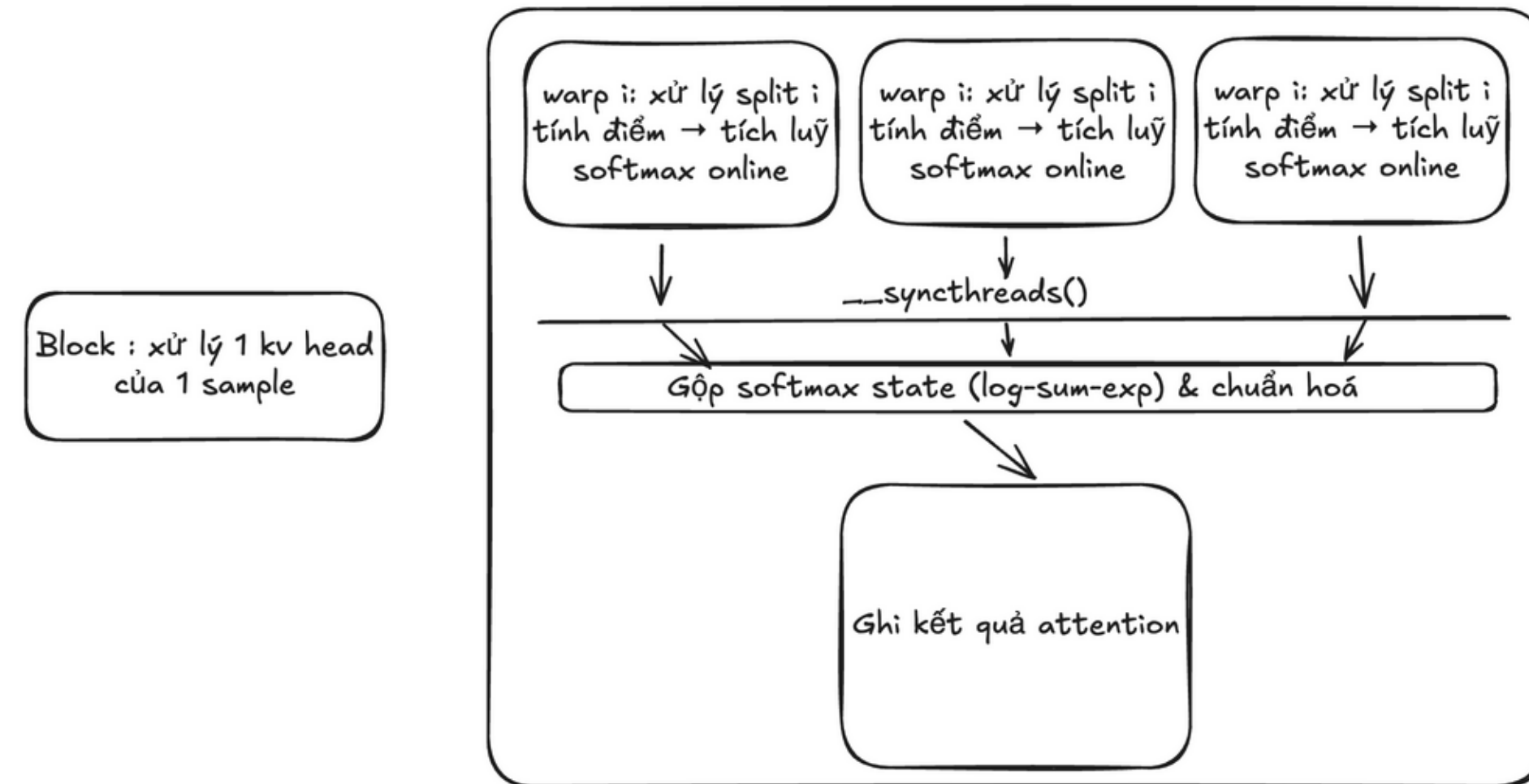
Attention

- Flash decoding:
 - Parallelizes by sequence length (divide into splits)
 - Online softmax: ensures numerically stable and merges softmax results from splits to achieve final attention result



Attention

- 1 warp = 8 heads $\times$ 8 lanes/head, D=64
 $\Rightarrow$ each lane handle 8 elements
- Read K/V (bf16) directly from global memory, bypassing shared memory:
 - Save staging cost into LDS
 - At the same time, all 8 heads in a warp read 64 elements from a same K/V row (fixed kv_h). Only the first read accesses global memory Lần đọc đầu tiên đi global, the rest hit cache.
 - Vectorize load Q,K,V, vectorize store output (gained 10% throughput).



Mixture of Experts

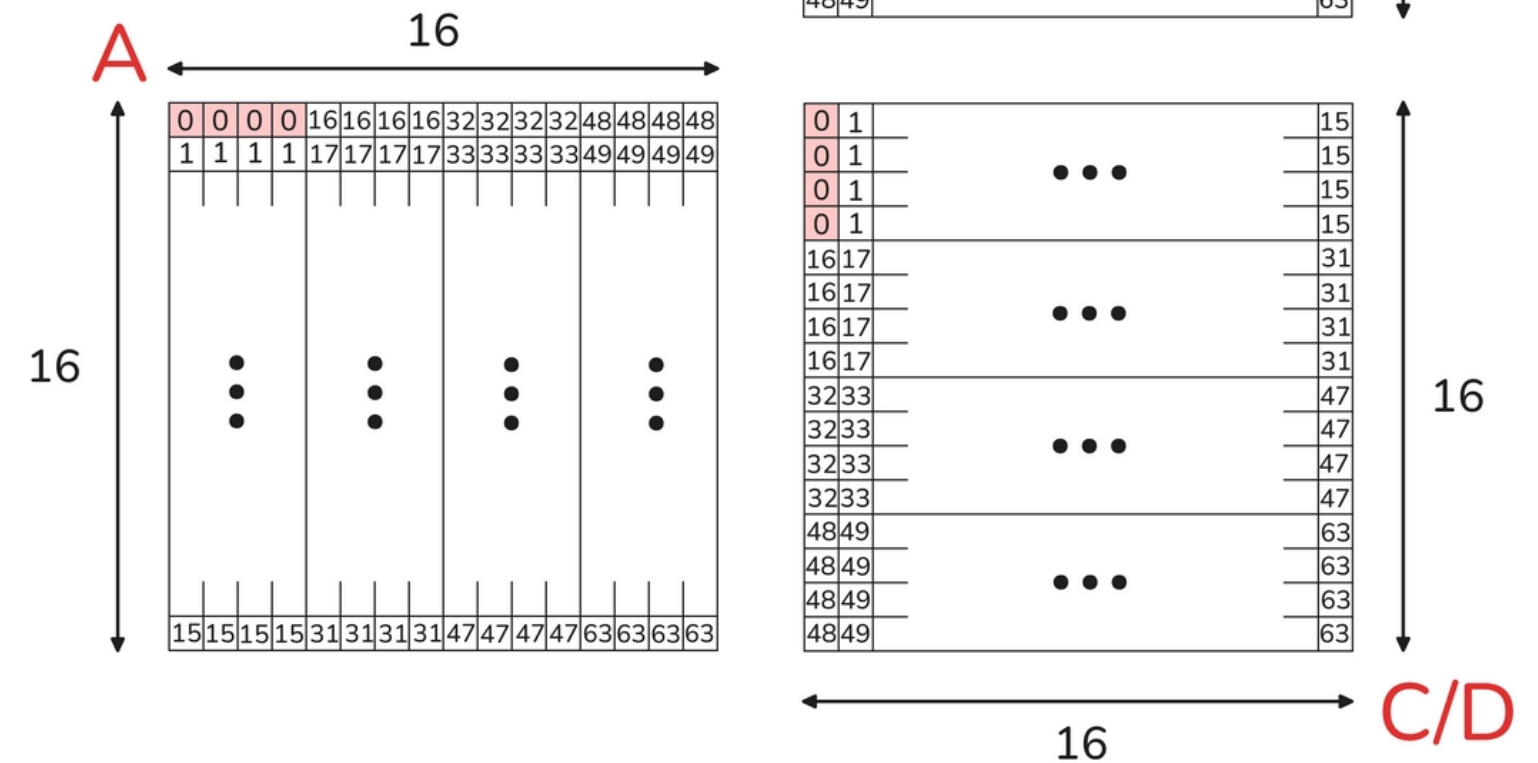
- Duplicate inputs by expert_token, reorder that each expert takes a continuous range of inputs
- Fused MLPs (matmul): ensure each block only handle 1 expert
- Two separate MLPs for each cases:
 - $\max(M) > 900$: 32x32x8
 - Otherwise: 16x16x16
- Pre-dequantize mlp weight for each layer before mlp & overlap with inference stream using ping-pong → reduce overhead by dequantizing MXFP4 directly inside mlp kernels

Matmul

`__builtin_amdgcn_mfma_f32_16x16x16bf16_1k`

- Perform: $C[16 \times 16] += A[16 \times 16] * B[16 \times 16]$ on 1 wavefront
- Higher throughput compared to instructions using input fp32 (eg: `16x16x4fp32`), but lower accuracy

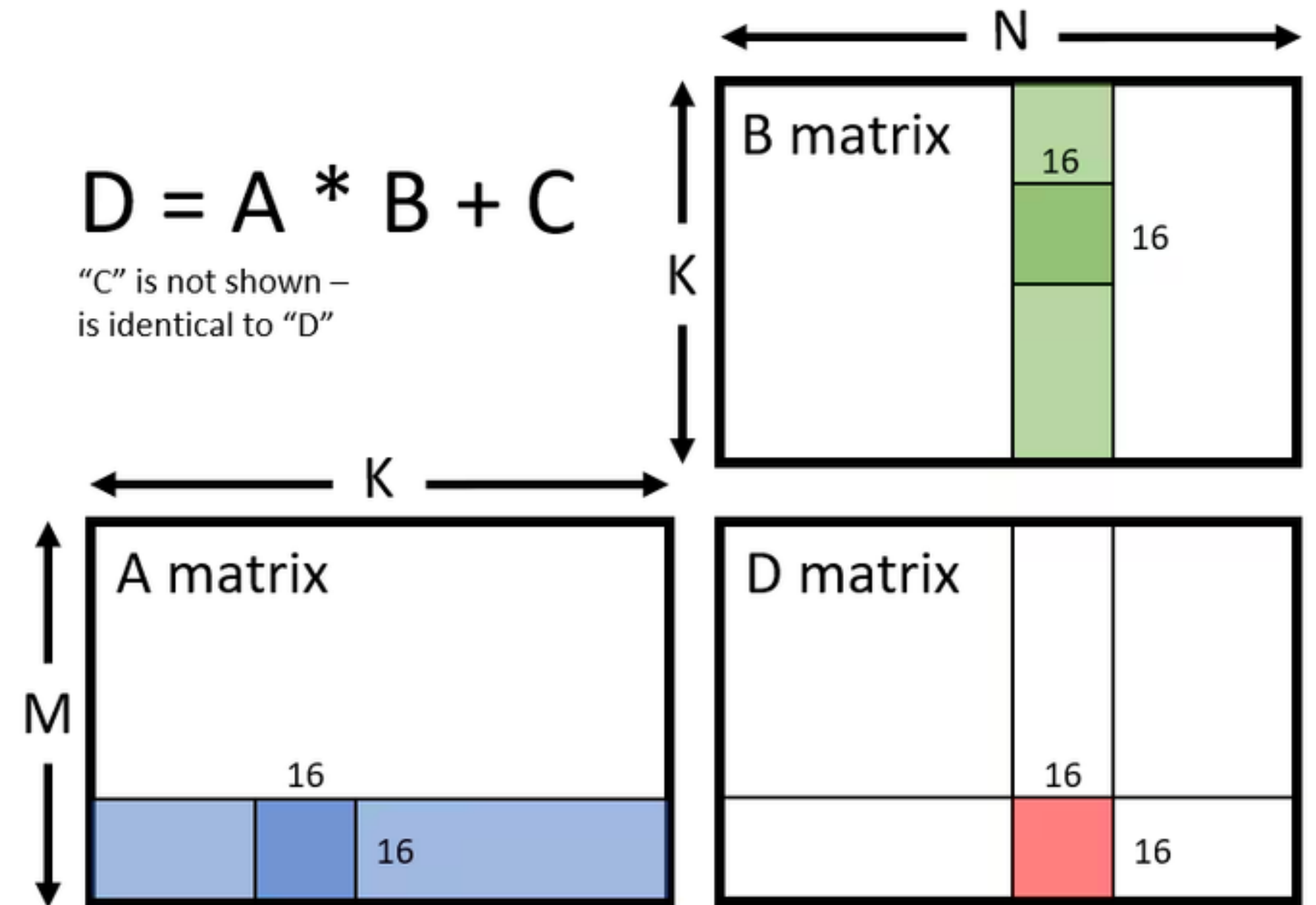
`v_mfma_f32_16x16x16_f16`



Matmul

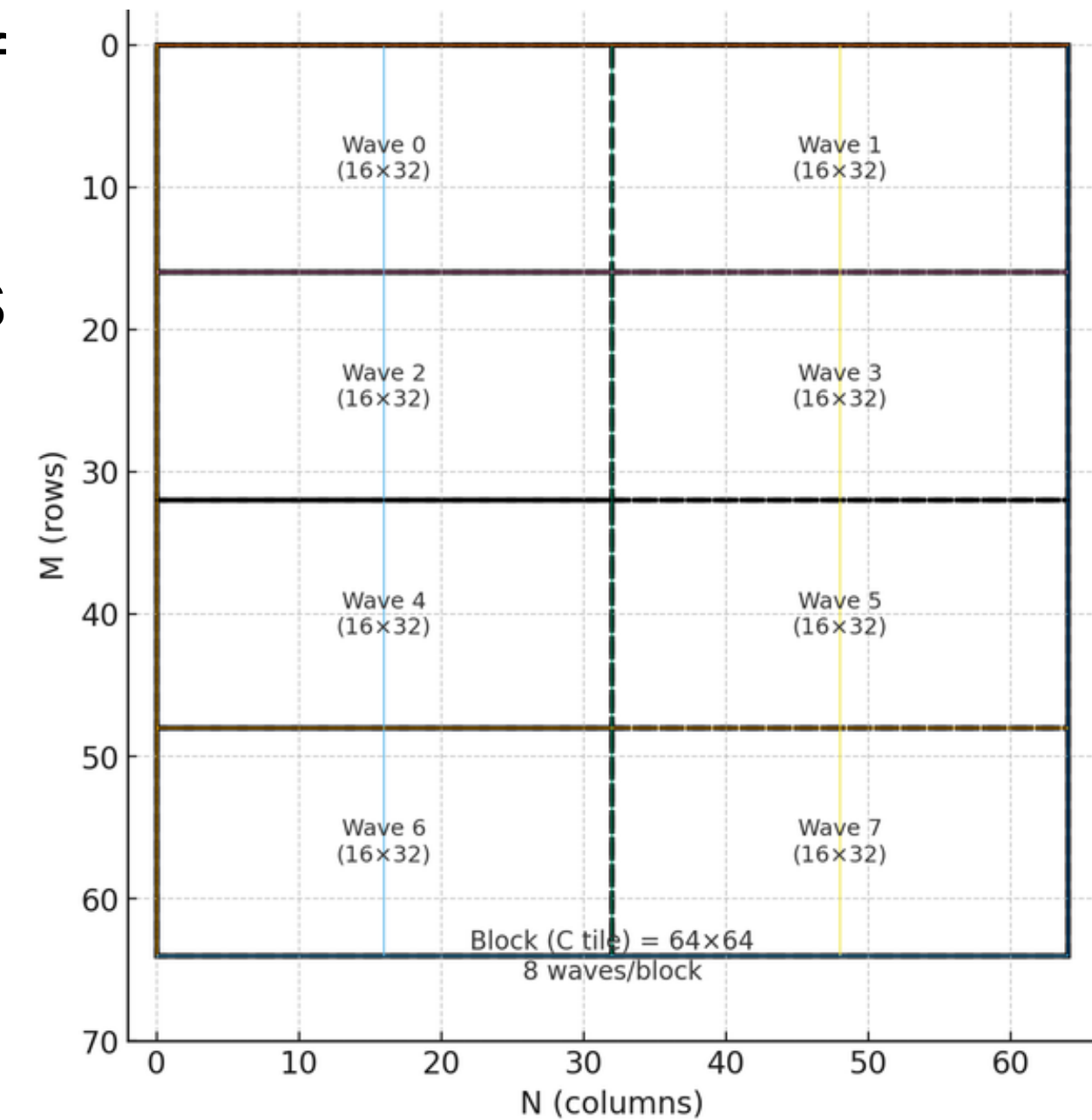
`__builtin_amdgcn_mfma_f32_32x32x8bf16_1k`

- Perform: $C[32 \times 32] += A[32 \times 8] * B[8 \times 32]$ on 1 wavefront
- Higher throughput compared to $16 \times 16 \times 16 \text{bf16}$ in case $M, N \gg K$, by better arithmetic intensity
- Each thread loads (32×4) tile A, B, stores (32×16) tile C



Matmul

- **Block tiling:** Each thread processes a 64×64 tile of C matrix
- **Wavefront tiling:** Inside each block, 8 wavefronts (64 threads/ wavefront), each handles a 16×32 tile of C.
- **K tiling:** Increase compute per load but require more LDS.
- Data flow: global memory $\rightarrow$ shared memory $\rightarrow$ registers.



Matmul

- Store W in row-major (better memory access pattern) → **$B = A * W^T$**
- Optimize global memory access: each thread reads an aligned continuous chunk: 16B (4 fp32/ 8 bf16) → utilize maximum memory bandwidth
- A, B are stored in bf16 format → load faster than when keep A in fp32
- Add padding to eliminate bank conflict
- Benchmark and use 32x32x8 and 16x16x16 for appropriate matrix sizes

Misc

- Move sampler from CPU to GPU → save transfer time and CPU computation cost
- Fuse small kernel: matmul + bias
- Optimize swiglu: fuse mlp+bias instead of bias+swiglu

Accuracy metrics: METEOR, BERTscore

Model	20B	120B
METEOR	0.385	0.304
BERTscore	0.96	0.96

Performance Metrics

```
Weights copied. w_qkv and out are transposed in-place on GPU ([K,N] layout).  
Weights copied. w_qkv and out are transposed in-place on GPU ([K,N] layout).  
Weights copied. w_qkv and out are transposed in-place on GPU ([K,N] layout).  
Weights copied. w_qkv and out are transposed in-place on GPU ([K,N] layout).  
Weights copied. w_qkv and out are transposed in-place on GPU ([K,N] layout).
```

```
warm up elapsed time(s): 106.496000
```

```
elapsed time(s): 261.126000, achieved throughput TPS (tok/s): 46793.613045
```

```
finish elapsed time(s): 0.081000
```

```
Weights copied (TP optimized): per-layer strided-gather tiling for MLP1/MLP2, biases BF16.  
Weights copied (TP optimized): per-layer strided-gather tiling for MLP1/MLP2, biases BF16.  
Weights copied (TP optimized): per-layer strided-gather tiling for MLP1/MLP2, biases BF16.  
Weights copied (TP optimized): per-layer strided-gather tiling for MLP1/MLP2, biases BF16.
```

```
warm up elapsed time(s): 289.563000
```

```
elapsed time(s): 712.407000, achieved throughput TPS (tok/s): 20549.732105
```

```
finish elapsed time(s): 0.230000
```

Future works

- Use fp16, of which native operations (add, mul) are supported by hardware, instead of bf16, which requires upcasting and downcasting
- Implement more mfma matmul kernels and benchmark carefully, especially for mlps